STRIPE PROJECT

1. OLTP Data System

D8.1. OLTP SQL Queries Examples

> *Nicolas Pichon --- AIA RNCP 38777 / BC02 / D8.1 : OLTP Queries / v36 --- 2026/11/05.*

D8.1.1. Contexte

Ce document propose quelques exemples de requête SQL sur le modèle transactionnel. Ces requêtes illustrent le chemin de paiement et le domaine d'exploitation courante. Elles s'appuient exclusivement sur les index du modèle physique - conçus spécifiquement pour le domaine transactionnel.

Le modèle physique de référence est défini dans l'annexe B [D2-B]. Le SGBD cible est *PostgreSQL* (16.15).

Domaine exploré

On n'explore pas les domaines exigés dans le cas d'étude [D0] : analyse de revenu, détection de fraude, segmentation client, dans la mesure où ces domaines sont *analytiques* et non transactionnels. Une requête *transactionnelle* s'exprime en quelques lignes identifiées par une clé ou un index sélectif et s'exécute avec la garantie d'une latence faible, de l'ordre de la milliseconde.

Synopsis :

#	Requête	Accès principal
D8.1.2.1	Lire une transaction	clé primaire `transaction_id`
D8.1.2.2	Dernières transactions d'un client	`ix_transaction_customer_created (customer_id, created_at)`
D8.1.2.3	Enregistrer un paiement (idempotent)	`IdempotencyKey` , plan en vigueur
D8.1.2.4	Abonnements à prélever	`ix_subscription_due` (index partiel défini sur place)
D8.1.2.5	Demandes RGPD proches de l'échéance	`ix_dsr_due (due_at)`
D8.1.2.6	Historique des accès aux données d'un client	`ix_dal_customer_occurred (customer_id, occurred_at)`

Conventions

- *Dates* : UTC de type `timestamptz` .
- *Montants* : entiers en *unités mineures* de la devise (la précision par rapport à la devise est donnée par `decimals` dans `Currency`).
  - Les requêtes renvoient des unités mineures, sauf mention contraire. Exemple : "5000 | EUR" signifie "50,00 euros".
- *Paramètres* : la syntaxe `:<nom>` désigne un paramètre de requête de nom `<nom>` (ex : `:merchant_id` , `:customer_id` , etc.).

D8.1.2. Requêtes transactionnelles

D8.1.2.1. Lire une transaction

Consigne : Retrouver une transaction à partir de son identifiant.

- Paramètre de la requête : l'identifiant de la transaction : `:transaction_id` .

```
SELECT t.transaction_id, t.merchant_id, t.customer_id, t.currency_code, t.amount, t.fee_amount,
       t.status_code, t.created_at
FROM Transaction t
WHERE t.transaction_id = ;
```

> *Commentaire* : Accès par la seule clé primaire `transaction_id` (pas de partitionnement).

> *Exemple de sortie* :

transaction_id	merchant_id	customer_id	amount	fee_amount	currency_code	status_code	created_at
[...]5bc72f	[...]0a4535	[...]84b05a	3457	134	EUR	successful	2026-10-12 09:13:47[...]

D8.1.2.2. Dernières transactions d'un client

Consigne : Récupérer les transactions les plus récentes d'un client.

- Paramètres : l'identifiant du client et la date limite la plus récente de la période couverte (pour permettre une pagination basée sur la date de création).

```
SELECT t.transaction_id, t.currency_code, t.amount, t.status_code, t.created_at
FROM Transaction t
WHERE t.customer_id = :customer_id AND t.created_at < :before
ORDER BY t.created_at DESC
LIMIT 20;
```

> *Commentaire* : le résultats de la requête est paginé *par curseur* par paquet de 20 transactions : la page suivante part du `created_at` de la dernière ligne de la page précédente, la première page `now()`. La pagination par curseur parcourt l'index `(customer_id, created_at)` à rebours et s'arrête après 20 lignes : son coût ne dépend pas de la taille de l'historique du client.

> *Exemple de sortie* :

transaction_id	created_at	amount	currency_code	status_code
[...]8ad432	2026-10-23 10:43:15[...]	1200	EUR	pending
[...]b6d74b	2026-10-12 09:33:47[...]	17457	EUR	suspended
[...]5bc72f	2026-10-12 09:13:47[...]	3457	EUR	successful

D8.1.2.3. Enregistrer un paiement (idempotent)

Consigne : Enregistrer un paiement ponctuel en garantissant son idempotence (ne pas créer deux transactions identiques lors d'un rejeu de la même requête).

- Paramètres : clé d'idempotence, identifiant de la transaction, du client, du produit, du moyen de paiement, de la devise, montant, etc. (toutes les variables définissant la transaction, sauf l'identifiant du marchand, qui est déduit de celui du client).

```
-- 1. Réserver la clé d'idempotence
---- La clé d'entête http 'idempotency-key' est fournie par l'appelant.
---- L'identifiant de transaction de type uuid v7 est fourni par l'application.
---- Si la clé (merchant_id, idempotency_key) n'existe pas dans la table,
---- la ligne est insérée et la requête renvoie l'id de transaction.
---- Sinon, la clé existe déjà : l'instruction DO NOTHING indique de ne rien faire : pas d'insertion, pas de
retour (ligne vide).
---- Noter que la requête renvoie également une ligne vide si le client n'existe pas.
INSERT INTO IdempotencyKey (merchant_id, idempotency_key, transaction_id)
SELECT c.merchant_id, :idempotency_key, :transaction_id
FROM Customer c
WHERE c.customer_id = :customer_id
ON CONFLICT (merchant_id, idempotency_key) DO NOTHING
RETURNING transaction_id;
```

```
-- 2. Créer la transaction.
---- Si aucune ligne n'est insérée, cela signifie qu'il manque un plan tarifaire
---- ou une définition de frais fixe associé à la devise courante.
INSERT INTO Transaction (transaction_id, merchant_id, customer_id, payment_method_id, product_id,
pricing_plan_id,
                         currency_code, amount, fee_amount,
                         ip_geolocation, device_type,
                         status_code, created_at, updated_by, updater_type)
SELECT :transaction_id, c.merchant_id, c.customer_id, :payment_method_id, :product_id, p.pricing_plan_id,
:currency_code, :amount,
       stripe_fee(:amount, p.commission_rate, f.fixed_fee),
       :ip_geolocation, :device_type,
       'pending', :created_at, :actor_id, :actor_type_code
FROM Customer c
JOIN PricingPlan p ON p.merchant_id = c.merchant_id
        AND p.effective_from <= :created_at::date AND (p.effective_to IS NULL OR p.effective_to >
:created_at::date)
JOIN PricingPlanFee f ON f.pricing_plan_id = p.pricing_plan_id AND f.currency_code = :currency_code
WHERE c.customer_id = :customer_id;
```

> *Commentaires*

- Les instructions SQL ci-dessus s'exécutent séquentiellement dans une transaction SQL.
- La commission *Stripe* est calculée dans la devise courante par la fonction applicative `stripe_fee` en utilisant le plan tarifaire en vigueur à la date de la transaction :
  - stripe_fee(amount, cr, ff) = round(amount * cr) + ff
- Les clés étrangères composites vérifient, sans code applicatif, que le moyen de paiement appartient au client, que le produit appartient au marchand, que le plan appartient au marchand et qu'il porte un frais fixe pour la devise. L'unicité de la clé d'idempotence est portée par la clé primaire `(merchant_id, idempotency_key)` de `IdempotencyKey`, table opérationnelle séparée de `Transaction` : elle se purge sur sa fenêtre de rejeu sans toucher aux transactions.

> *Exemple de sortie* :

- Scénario : création réussie d'une transaction suivie d'un rejeu avec transaction modifiée.
- Plan tarifaire du marchand : plan en vigueur à la date courante, avec un taux de commission de 3% et un frais fixe de 30 EUR (non défini en BHD (dinar de Bahreïn)).
- Paramètres de la transaction :

`:idempotency_key`	order-1042-attempt-1
`:transaction_id`	01a16809-595a-7aee-bca5-65d07b0d186c
`:customer_id`	[...]84b05a
`:payment_method_id`	[...]1ea0ac (moyen de paiement de ce client)
`:product_id`	[...]90c688 (produit du marchand du client)
`:amount`	5000 (centimes)
`:currency_code`	EUR
`:ip_geolocation`	203.0.113.9
`:device_type`	mobile
`:created_at`	`now()` au moment de l'exécution
`:actor_id`	api-key-42
`:actor_type_code`	merchant_api

- Résultats :

#	Étape	Résultat
1.	Réservation de la clé	`[...]0d186c`
2.	Création de la transaction	`INSERT 0 1`
		(1 ligne insérée)
3.	Rejeu : réservation	∅
	(même clé, nouvel identifiant : `[...]e26dd3`)	(clé existante)
4.	Rejeu : création	`INSERT 0 0`
	(devise redéfinie en BHD = frais fixe inconnu)	(0 ligne insérée)

- Contrôle de la ligne créée à l'étape 2 :

transaction_id	amount	fee_amount	currency_code	status_code
[...]0d186c	5000	180	EUR	pending

D8.1.2.4. Abonnements à prélever

Consigne : Réserver un lot d'abonnements arrivés à échéance pour un worker de facturation.

```
SELECT s.subscription_id, s.customer_id, s.payment_method_id, s.amount, s.currency_code, s.current_period_end
FROM Subscription s
WHERE s.status_code IN ('active', 'past_due') AND s.current_period_end <= now()
ORDER BY s.current_period_end
LIMIT 500
FOR UPDATE SKIP LOCKED;
```

> *Exemple de sortie* :

subscription_id	customer_id	payment_method_id	amount	currency_code	current_period_end
[...]e64ec7	[...]a0b693	[...]907a8d	4700	EUR	2026-10-20 07:57:46[...]
[...]562291	[...]6ca9f2	[...]9f52ab	1200	EUR	2026-10-22 11:57:46[...]
[...]94979d	[...]af7c85	[...]513fa9	4700	EUR	2026-10-23 06:45:46[...]

> *Commentaires* :

- L'instruction `SKIP LOCKED` permet à plusieurs workers de travailler en parallèle sans se bloquer.
- Le prédicat reprend celui de l'index partiel suivant (à rajouter au modèle hors périmètre) :

```
CREATE INDEX ix_subscription_due ON Subscription (current_period_end)
WHERE status_code IN ('active','past_due');
```

Cet index ne contient que les abonnements susceptibles d'être prélevés.

D8.1.2.5. Demandes d'exercice de droits proches de l'échéance

Consigne : Repérer les demandes d'exercice de droits non terminées dont l'échéance légale tombe dans les cinq jours (ou est dépassée).

```
SELECT r.request_id, r.merchant_id, r.request_type, r.status_code,
       r.due_at, r.due_at - now() AS time_left
FROM DataSubjectRequest r
JOIN DataSubjectRequestStatus rs ON rs.status_code = r.status_code
WHERE NOT rs.is_terminal AND r.due_at < now() + INTERVAL '5 days'
ORDER BY r.due_at;
```

> *Exemple de sortie* :

request_id	merchant_id	request_type	status_code	due_at	time_left
[...]50bc40	[...]0a4535	erasure	in_progress	2026-10-21 11:44:24[...]	-2 days
[...]fbf204	[...]0a4535	access	received	2026-10-23 17:44:24[...]	06:00:00
[...]27bf34	[...]0a4535	rectification	received	2026-10-26 11:44:24[...]	3 days 01:00:00

D8.1.2.6. Historique des accès aux données d'un client

Consigne : Répondre à la question « qui a consulté les données de ce client, et quand ? ».

- Paramètres : l'identifiant du client (`:customer_id`) et la date limite la plus ancienne de la période couverte (`:since`).

```
SELECT l.occurred_at, l.actor_id, l.actor_type_code, l.action_code, l.resource_type, l.was_authorized
FROM DataAccessLog l
WHERE l.customer_id = :customer_id AND l.occurred_at >= :since
ORDER BY l.occurred_at DESC;
```

> *Exemple de sortie* :

occurred_at	actor_id	actor_type_code	action_code	resource_type	was_authorized
2026-10-23 09:41:04[...]	analyst-nora	user	update	Customer	true
2026-10-11 11:37:14[...]	svc-billing	system	read	PaymentMethod	true
2026-09-23 10:43:21[...]	intern-07	user	export	Customer	false

---

Couverture des exigences

#	Exigence [D0]	Couverture	État
BR1	intégrité transactionnelle	D8.1.2.3 (idempotence) ;	🟠 (à étendre)
		intégrité vérifiée par FK composites	
BR1/TR1/OLTP	latence minimale	accès par clé ou par index sélectif	🟢
BR5/TR5	conformité, audit	D8.1.2.5 (échéances RGPD) ;	🟢
		D8.1.2.6 (qui accède à quoi)	

Références

Spécifications

- [D0] Stripe Project : Introduction

Annexes

- [D2-B] OLTP Physical Model